Branch and Loop Handling in the APARA C Compiler

How C control flow is broken down into APARA mcode instructions

APARA C Compiler Project

June 22, 2026

1 Introduction and Strategy

This report explains how the compiler lowers every C control-flow construct — `if/else`, `while`, `do-while`, `for`, `switch`, `break/continue` — down to APARA mcode, across the same two compiler stages used for everything else: `ir_gen.py` (AST → three-address IR) and `codegen.py` (IR → mcode).

The single governing strategy is that the IR has no nested structure at all — no if/while/for nodes survive into it. Every construct is lowered, once and for all in `ir_gen.py`, into exactly three IR primitives: `IRLabel` (a named point in the flat instruction stream), `IRJump` (an unconditional jump to a label), and `IRCondJump` (jump to a label if a comparison holds). By the time `codegen.py` sees the program, an `if` statement and a `while` loop are built from *exactly the same two ingredients* — only the arrangement of labels and jumps differs. This is what makes a single, uniform mcode-level treatment (Section 5) possible: the bundler and the assembler never need to know what kind of C statement produced a given branch.

2 Lowering Each Construct: Algorithms and Flow

Each construct below is handled by its own `visit_Xxx` method in `ir_gen.py`, but all of them follow the same pattern: allocate a small, fixed set of labels up front, then emit `IRLabel` /`IRCondJump`/`IRJump` around a recursive call that lowers the nested body.

`if` / `if-else`

Listing 1: `visit_If`, ir_gen.py:724–735

```
function LOWER-IF(cond, ifTrueBody, ifFalseBody):
    t_lbl, e_lbl <- new labels "if_t", "if_e"
    if ifFalseBody exists:
        f_lbl <- new label "if_f"
        EMIT-COND(cond, true->t_lbl, false->f_lbl)
        LABEL t_lbl;  LOWER(ifTrueBody);  JUMP e_lbl
        LABEL f_lbl;  LOWER(ifFalseBody)
    else:
        EMIT-COND(cond, true->t_lbl, false->e_lbl)
        LABEL t_lbl;  LOWER(ifTrueBody)
    LABEL e_lbl
```

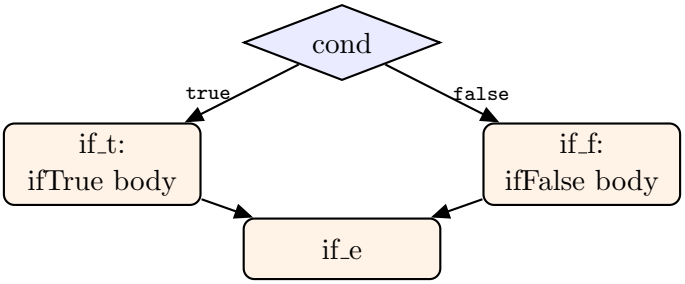

Figure 1: `if/else` flow. The `if_f` branch and its incoming edge disappear entirely when there is no `else`.

`while`

Listing 2: `visit_While`, ir_gen.py:737–746

```
function LOWER-WHILE(cond, body):
    cond_lbl, body_lbl, end_lbl <- new labels "wc", "wb", "we"
    push (break->end_lbl, continue->cond_lbl)
    LABEL cond_lbl
    EMIT-COND(cond, true->body_lbl, false->end_lbl)
    LABEL body_lbl;  LOWER(body);  JUMP cond_lbl
    LABEL end_lbl
    pop break/continue targets
```

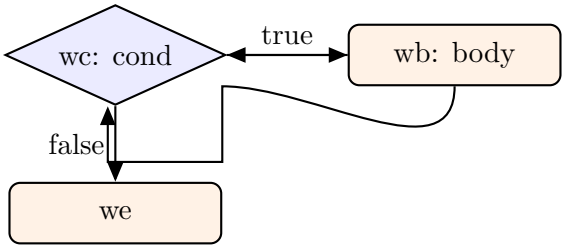

Figure 2: `while` flow: the body always jumps back to `wc`, which is the only place the condition is (re-)tested. `break` jumps straight to `we`; `continue` jumps straight to `wc`.

`do-while`

Structurally the same three labels as `while`, but the body label comes *first* and the condition is tested only after the first execution of the body — the condition check still lives at a single point, just after the body instead of before it (`visit_DoWhile`, ir_gen.py:748–756):

```
function LOWER-DO-WHILE(body, cond):
    body_lbl, cond_lbl, end_lbl <- new labels "dw_b", "dw_c", "dw_e"
    LABEL body_lbl;  LOWER(body)
    LABEL cond_lbl
    EMIT-COND(cond, true->body_lbl, false->end_lbl)
    LABEL end_lbl
```

`for`

A `for` loop is a `while` loop with two extra, fixed slots: the initializer runs once before the condition label, and the increment expression gets its *own* label so that `continue` can jump there without re-running the initializer (`visit_For`, ir_gen.py:758–774):

```
function LOWER-FOR(init, cond, incr, body):
    cond_lbl, body_lbl, incr_lbl, end_lbl <- new labels "fc", "fb", "fi", "fe"
    push (break->end_lbl, continue->incr_lbl)
    LOWER(init)                                    // runs exactly once
    LABEL cond_lbl
    if cond exists:  EMIT-COND(cond, true->body_lbl, false->end_lbl)
    else:             JUMP body_lbl                // no condition -> infinite loop
    LABEL body_lbl;  LOWER(body)
    LABEL incr_lbl;  LOWER(incr);  JUMP cond_lbl
    LABEL end_lbl
    pop break/continue targets
```

`switch`

APARA mcode has no jump-table instruction, so `switch` lowers to a linear chain of equality tests, evaluated top to bottom — algorithmically a sequence of `if` statements, not a single dispatch (`visit_Switch`, ir_gen.py:776–810):

```
function LOWER-SWITCH(value, cases, default, body):
    end_lbl <- new label "sw_end";  push break->end_lbl
    for each (caseValue, label) in cases:           // in source order
        diff <- value - caseValue
        if diff == 0:  goto label                    // IRCondJump per case
    if default exists:  JUMP default.label
    else:                JUMP end_lbl
    for each (label, stmts) in cases:  LABEL label;  LOWER(stmts)
    if default exists:  LABEL default.label; LOWER(default.stmts)
    LABEL end_lbl
    pop break target
```

Because there is no dispatch table, a `switch` with N cases costs N sequential comparisons in the worst case (the last `case` matched) — the same asymptotic cost as an equivalent `if`/`else if` chain, which is exactly what it compiles to.

`break` / `continue`

Trivial once the enclosing construct has pushed its targets: each is a single `IRJump` to whichever label is currently on top of a one-deep `_break_to`/`_cont_to` pair of fields (ir_gen.py:820–824), saved and restored around every loop/switch so nested constructs target the *innermost* enclosing one:

```
function LOWER-BREAK():     if break_target exists:  JUMP break_target
function LOWER-CONTINUE():  if continue_target exists:  JUMP continue_target
```

Label-naming convention, at a glance

3 Condition Lowering: From a C Expression to a Single Branch

Every construct above calls one shared helper, `_emit_cond` (ir_gen.py:1277–1284), to turn a C boolean expression into IR jumps. Its strategy has two paths:

Listing 3: `_emit_cond`, ir_gen.py:1277–1284

```
function EMIT-COND(cond, true_lbl, false_lbl):
    if cond is a direct comparison (>, <, >=, <=, ==, !=):
        l <- EVAL(cond.left);  r <- EVAL(cond.right)
```

Table 1: Label prefixes by construct (all generated by `ir_gen.py`'s `_lbl()`)

Construct	Labels generated
`if`	`if_t` (true body), `if_f` (else body, if present), `if_e` (join point)
`while`	`wc` (condition check), `wb` (body), `we` (exit)
`do-while`	`dw_b` (body), `dw_c` (condition check), `dw_e` (exit)
`for`	`fc` (condition), `fb` (body), `fi` (increment), `fe` (exit)
`switch`	`case` (one per `case` label), `dflt` (default), `sw_end` (exit)
`&&`/`||`	`andF`/`andE`, `orT`/`orE` (short-circuit result materialization)
bare comparison	`cT`/`cE` (only when a comparison is used as an ordinary 0/1-valued expression, not as a direct branch condition)

```
        emit IRCondJump(l, cond.op, r, true_lbl, false_lbl)        // fast path
        return
    v <- EVAL(cond)                                                 // general
    expression
    emit IRCondJump(v, '!=', 0, true_lbl, false_lbl)                // "is it truthy
    ?"
```

The **fast path** matters because APARA's actual hardware branch instruction, `$goto`, only ever tests *one register against zero* (with one of six relational operators) — it has no two-register-compare form. So whenever a condition is already a direct comparison (the overwhelmingly common case — every loop and `if` in this report's examples), `_emit_cond` skips materializing a 0/1 boolean value entirely and emits the comparison directly as an `IRCondJump`. Only when a condition is some other kind of expression (a bare variable, a function call result, a pre-computed `&&`/`||`) does it fall back to "evaluate it, then test the result against zero."

`codegen.py`'s `_emit_cond_branch` (codegen.py:463–505) then has to bridge the gap between the IR's general two-operand comparison and the hardware's one-register-against-zero branch:

Listing 4: `_emit_cond_branch`, codegen.py:463–505 (essence)

```
function EMIT-BRANCH(left, op, right, true_lbl, false_lbl):
    if left and right are both compile-time constants:
        fold the comparison now; emit one unconditional jump; return    // constant
    folding
    if right == 0:
        if op in {<, <=}:  scratch <- 0 - left;  branch on (scratch, flip(op))
        else:               branch directly on (left, op)
    else:
        scratch <- (left - right)   if op in {>,>=,==,!=}
                   (right - left)   if op in {<,<=}     // sign flip trick
        branch on (scratch, op-or-flipped-op)
    if false_lbl exists:  emit unconditional JUMP false_lbl
```

Every two-operand comparison is reduced to a *subtraction* followed by a single register-vs-zero branch — the same "synthesize the missing operation from a cheaper primitive" strategy already seen elsewhere in this compiler (e.g. `%` synthesized as `a - (a/b)*b`). An `IRCondJump` with no false label at all (the common case inside a loop, where falling through to the next label *is* the false branch) skips the trailing unconditional jump — but every `IRCondJump` that specifies *both* a true and a false label, as every `visit_While`/`visit_For` condition does, always compiles to exactly **two** consecutive control-transfer instructions: one conditional, one unconditional.

4 A Fully Worked, Hardware-Verified Example

Listing 5: Source

```c
int main() {
    int i;
    i = 0;
    while (i < 16) {
        i = i + 1;
    }
    return i;
}
```

Listing 6: Generated IR (the loop only) — exactly the `wc/wb/we` pattern from Section 2

```
wc_1:
  _t2 = &stack[FP-8]
  _t3 = *(_t2+0)
  if _t3 < 16 goto wb_2          (* IRCondJump: true->wb_2, false->we_3, fast path *)
wb_2:
  _t4 = &stack[FP-8]
  _t5 = *(_t4+0)
  _t6 = _t5 + 1
  _t7 = &stack[FP-8]
  *(_t7+0) = _t6
  goto wc_1
we_3:
```

Listing 7: Resulting bundled mcode (loop only) — 9 bundles, hardware-verified

```
wc_1:
||
    + $r2 ($i64) $r26 -8          // r2 = &i
;
||
    $ld ($i32) $r3 [$r2 + 0]      // r3 = i
    + $r4 ($i64) $r0 16           // r4 = 16
;
||
    - $r5 ($i64) $r4 $r3          // r5 = 16 - i  (sign-flip trick for '<')
;
||
    ? ($i64) $r5 > $goto wb_2     // branch TRUE: 16-i > 0  <=>  i < 16
;
||
    ? ($i64) $r0 == $goto we_3    // branch FALSE: unconditional fall-through to exit
;
wb_2:
||
    + $r4 ($i64) $r26 -8
;
||
    $ld ($i32) $r5 [$r4 + 0]
;
||
    + $r6 ($i64) $r5 1            // r6 = i + 1
    + $r7 ($i64) $r26 -8
;
||
    $st ($i32) [$r7 + 0] $r6      // store i+1 back
    ? ($i64) $r0 == $goto wc_1    // loop-back -- SHARES a bundle with the store
```

```
32 ;
33 we_3:
```

Two details worth calling out explicitly, both confirmed directly from this listing:

- The condition `i < 16` becomes a *subtraction* (`$r5 = $r4 - $r3`, i.e. `16 - i`) followed by a branch on `$r5 > 0` — exactly the sign-flip strategy from `EMIT-BRANCH` above, because APARA's `$goto` cannot compare two registers directly.
- The loop-back jump (`? $r0 == $goto wc_1`) does *not* need its own bundle: it shares one with the preceding `$st`, because the bundler's only rule is that a control-transfer instruction must be **last** in its bundle, not that it must be **alone** in it (Section 5).

5 mcode-Level Handling: One Instruction, Two Hard Rules

By the time control flow reaches mcode, every branch in this report — regardless of which C construct produced it — is one of exactly two instruction shapes:

- **Conditional**: `? ($iN) $reg OP $goto label` — six possible `OP`s (`>`, `<`, `>=`, `<=`, `==`, `!=`), always comparing one register against an implicit zero.
- **Unconditional**: `? ($i64) $r0 == $goto label` — there is no dedicated unconditional-jump instruction on this ISA, so the compiler reuses the conditional form with the always-true tautology `0 == 0`, needing no extra register at all (`_emit_jump`, codegen.py:459–461).

`bundler.py` enforces two rules specific to these instructions (Section 2.4 of the main project report covers the full hazard-detection algorithm; only the control-flow-specific pair is repeated here):

1. **A control-transfer instruction must be the last instruction in its bundle.** Once one has been added to the bundle being packed, the very next instruction — whatever it is — forces a new bundle to start. It does *not* need to be the *only* instruction, as Section 4's loop-back example shows directly.
2. **A label forces a new bundle boundary.** Every label (`wc_1`, `wb_2`, `we_3`, ...) attaches to the start of whatever bundle comes right after it; two different labels can collapse onto the very same bundle only if nothing meaningful separates them, in which case `_merge_duplicate_labels` rewrites every reference to the dropped label so it points at the surviving one (this is the exact mechanism — and the exact bug, before it was fixed — behind the duplicate-label assembler crash documented in the project's main bug report).

6 Summary

Every C control-flow construct — however different it looks in source — is reduced by `ir_gen.py` to the same three IR primitives (`IRLabel`, `IRJump`, `IRCondJump`), arranged in a construct-specific but fixed pattern of labels. `codegen.py` then reduces every two-operand comparison to a subtraction plus a single register-against-zero test, because that is the only branch form APARA's hardware actually supports, and reduces unconditional jumps to a tautological comparison, because APARA has no dedicated unconditional-jump instruction at all. The bundler's only control-flow-specific obligations are to keep each control-transfer last in its bundle and to treat every label as a hard bundle boundary — everything else (how many bundles a loop costs, what shares a bundle with what) falls out of the same general hazard-packing algorithm used for arithmetic and memory instructions throughout this compiler.